Алгоритмы обработки больших данных

L03. Batch mode

Апрель, 2026
Смирнов Сергей
smirnov.work@mail.ru
@serg_v_smirnov

Agenda

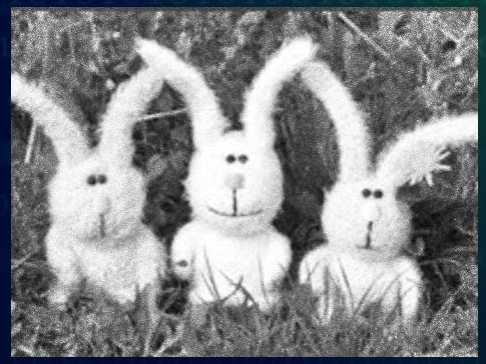
1. BigData intro
2. Stream & flink intro
3. ***Batch mode, pyflink runtime architecture***
4. DataStream operators API, keyed state
5. Stateful stream processing. Windows, Triggers, ProcessFunction, State, Timers, Watermarks
6. State and Chekpoint, State Backends. Table API и SQL API
7. CEP, CDC, ML, Metrics, Failure recovery, Execution configuration, Async IO, Delivery guarantees. Best (worst) practices
8. Flink-SQL, базовый синтаксис, работа с окнами
9. Flink-SQL, соединение потоков, udf, оптимизация
10. Whats new, trends (fluss, streamhouse)

Homework

Термины, которые полезно знать

- *broker*
- *key-value storage*
- *state, stateless vs statefull*
- *dag*
- *async*
- *latency & throughput*
- *поток*
- *гарантии доставки*
- *стриминг*
- *виды окон*

Unified API



~~Кролики это не только ценный мех~~
flink это не только стриминг, но и батч

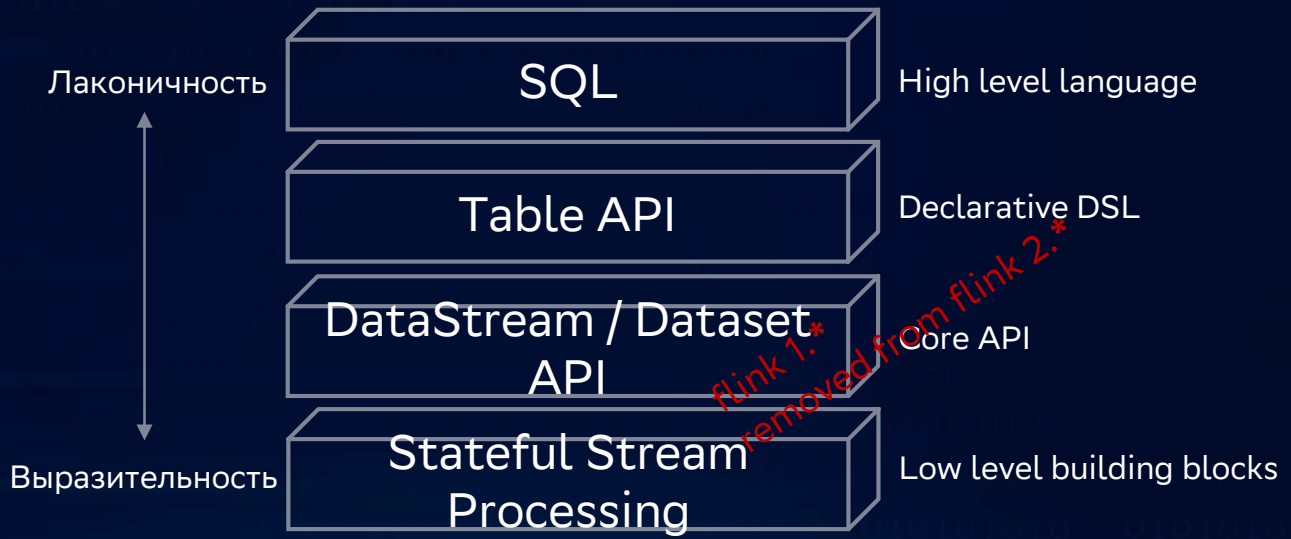
DataSet API — batch processing

DataStream API — stream processing
batch*

Table / SQL — batch&stream processing

flink 1.*
removed from flink 2.*

Batch processor	Stream processor
✓ bound execution	✓ bound execution
✗ unbound execution	✓ unbound execution



Requirements

Java 11

Support for Java 11 was added in 1.10.0.

The following Flink features have not been tested with Java 11:

- Hive connector
- Hbase 1.x connector

Untested language features

- Modularized user jars have not been tested.

Java 17

We use Java 17 by default in Flink 2.0.0 and is the recommended Java version to run Flink on. This is the default version for docker images.

These Flink features have not been tested with Java 17:

- Hive connector
- Hbase 1.x connector

Java 21

Experimental support for Java 21 was added in 2.0.0. (FLINK-33163)

These Flink features have not been tested with Java 21:

- Hive connector
- Hbase 1.x connector

Python

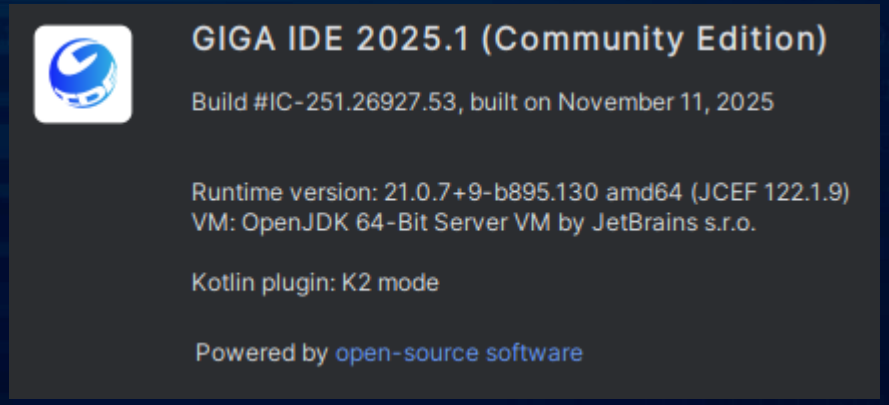
Supported versions 3.9, 3.10, 3.11, or 3.12. Python 3.8 and earlier versions are generally no longer supported in recent Flink releases (starting from Flink 2.2)

https://nightlies.apache.org/flink/flink-docs-stable/docs/deployment/java_compatibility/

https://nightlies.apache.org/flink/flink-docs-release-2.2/docs/dev/python/installation/

Working environment

GIGA IDE



Java 21

```
openjdk 21.0.10 2026-01-20
OpenJDK Runtime Environment (build 21.0.10+7-Ubuntu-124.04)
OpenJDK 64-Bit Server VM (build 21.0.10+7-Ubuntu-124.04, mixed mode, sharing)
```

Python

```
Python 3.12.3 (main, Mar  3 2026, 12:15:18) [GCC 13.3.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
```

Ubuntu

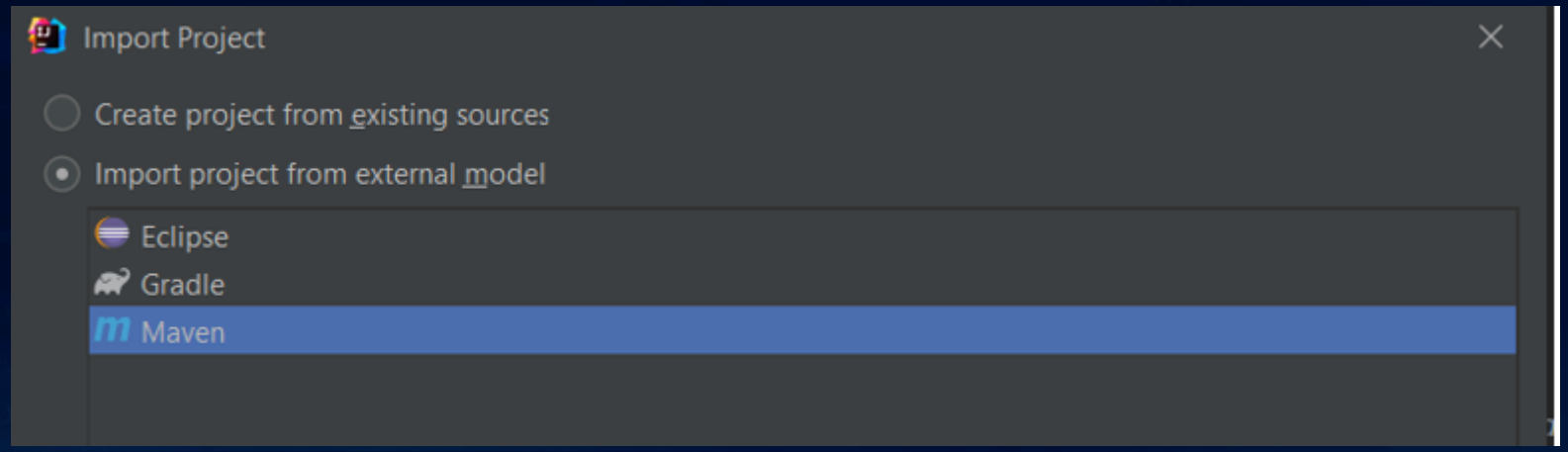
```
Distributor ID: Ubuntu
Description:    Ubuntu 24.04.4 LTS
Release:        24.04
Codename:       noble
```

Samurai

git clone https://github.com/apache/flink.git

cd flink

mvn clean package -DskipTests



Hello, world!

Batch, wordcount

```
final String resPath = System.getProperty("user.dir") + "\\assets\\data\\\\res.txt";
final ExecutionEnvironment env = ExecutionEnvironment.getExecutionEnvironment();

DataSource<String> ds = env.readTextFile(filePath);

AggregateOperator<Tuple2<String, Integer>> ds2 =
    ds.flatMap((FlatMapFunction<String, Tuple2<String, Integer>>) (value, out) -> {
      for (String word: value.split(" ")) {
        out.collect(new Tuple2<>(word, 1));
        }
      })
        .returns(Types.TUPLE(Types.STRING, Types.INT))
        .groupBy(0).sum(1);
ds2.print();
```

flink 1.*
removed from flink 2.*

Hello, world!

Batch, wordcount

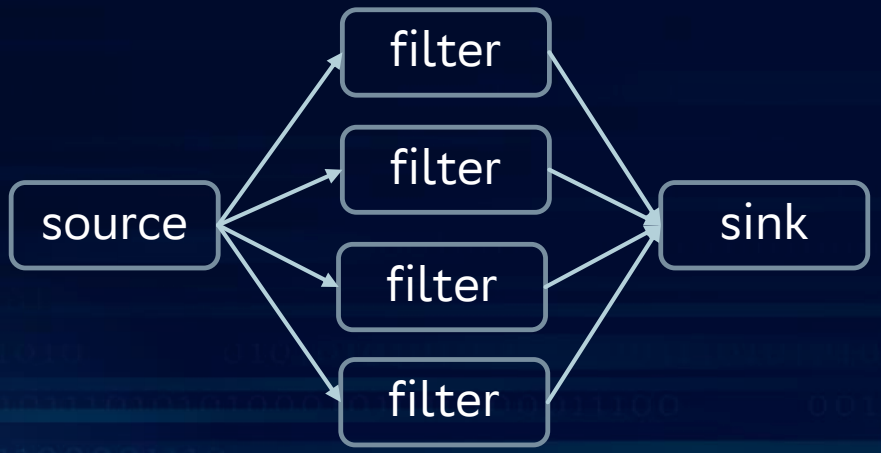
```
words_path = os.path.join(os.path.dirname(os.getcwd()), "assets", "data", "words.txt")
res_path = os.path.join(os.path.dirname(os.getcwd()), "assets", "data", "res.txt")


def batch_ex_1():  1 usage
    env = StreamExecutionEnvironment.get_execution_environment()
    env.set_runtime_mode(RuntimeExecutionMode.BATCH)
    ds = env.from_source(
        source=FileSource.for_record_stream_format(StreamFormat.text_line_format(),
                                                   *paths: words_path)
        .process_static_file_set().build(),
        watermark_strategy=WatermarkStrategy.no_watermarks(),
        source_name="words"
    )

    ds2 = (ds.flat_map(lambda val: [(word, 1) for word in val.split()])
           .key_by(lambda val: val[0])
           .sum(1)
           )
    ds2.print()
```

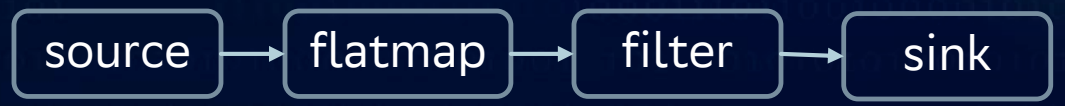
Physical & logical execution plan

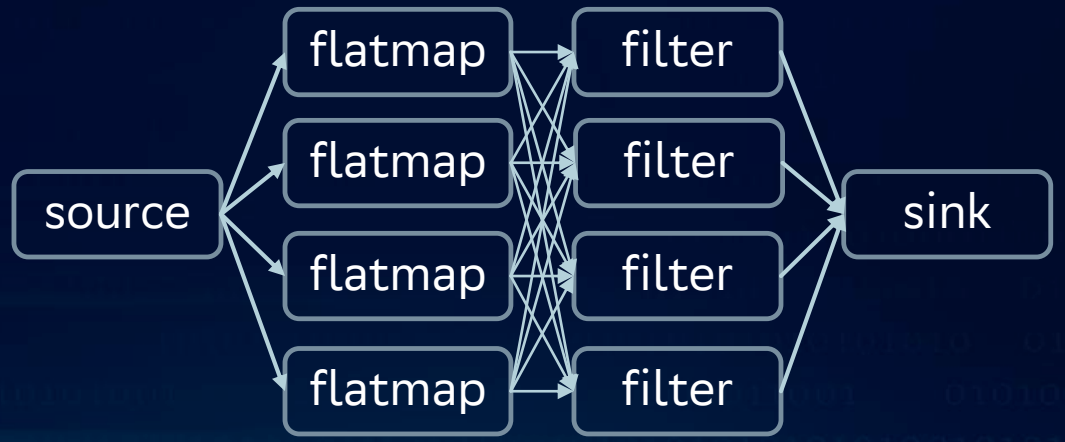
readTextFile(filePath)

print()

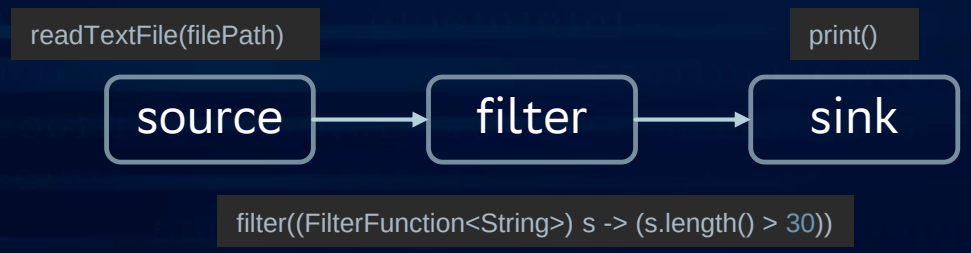

```
filter((FilterFunction<String>) s -> (s.length() > 30))
```

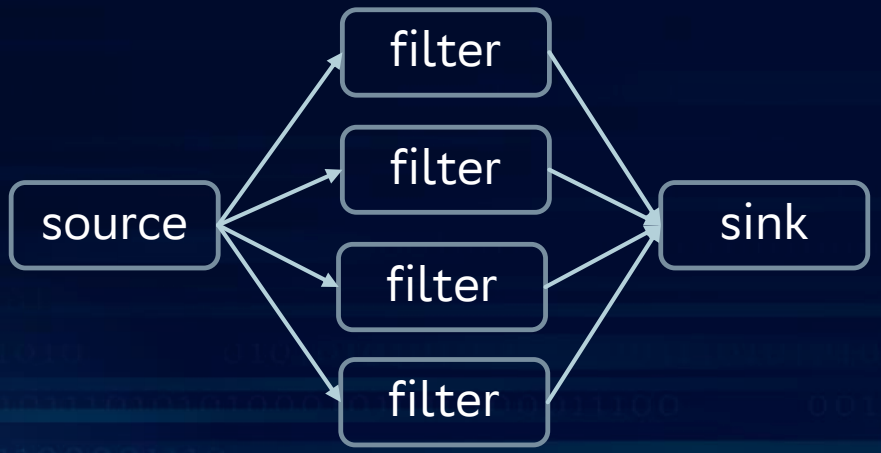


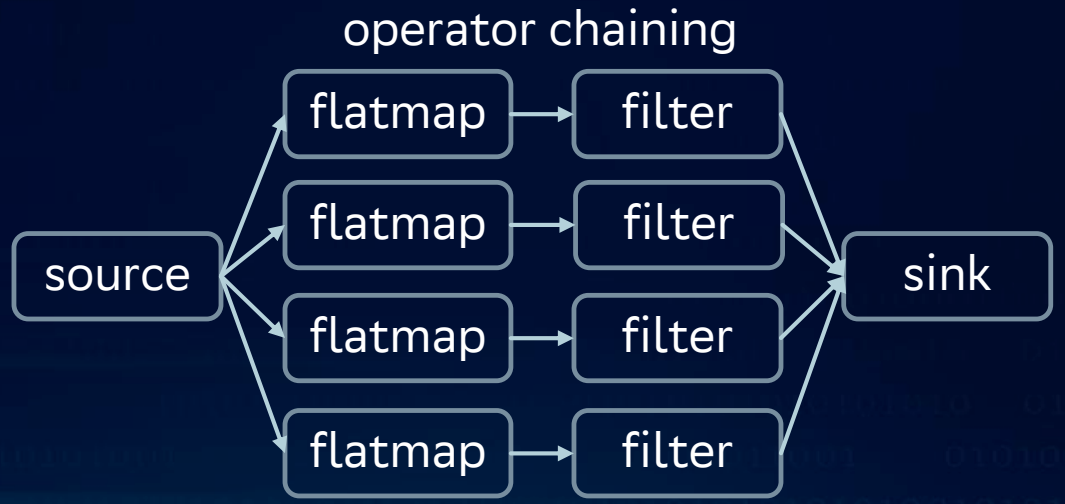




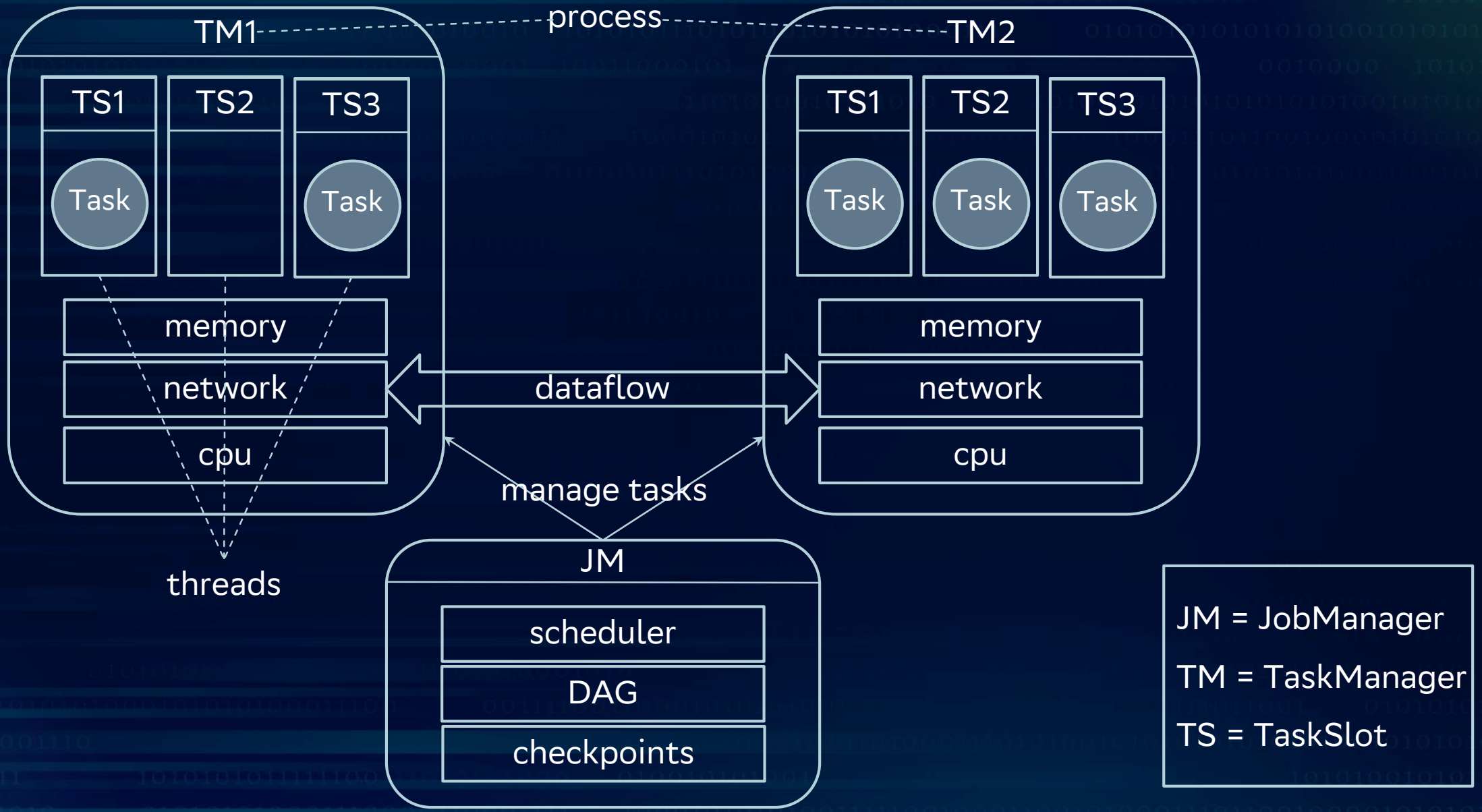
Physical & logical execution plan







Distributed execution



JM = JobManager

TM = TaskManager

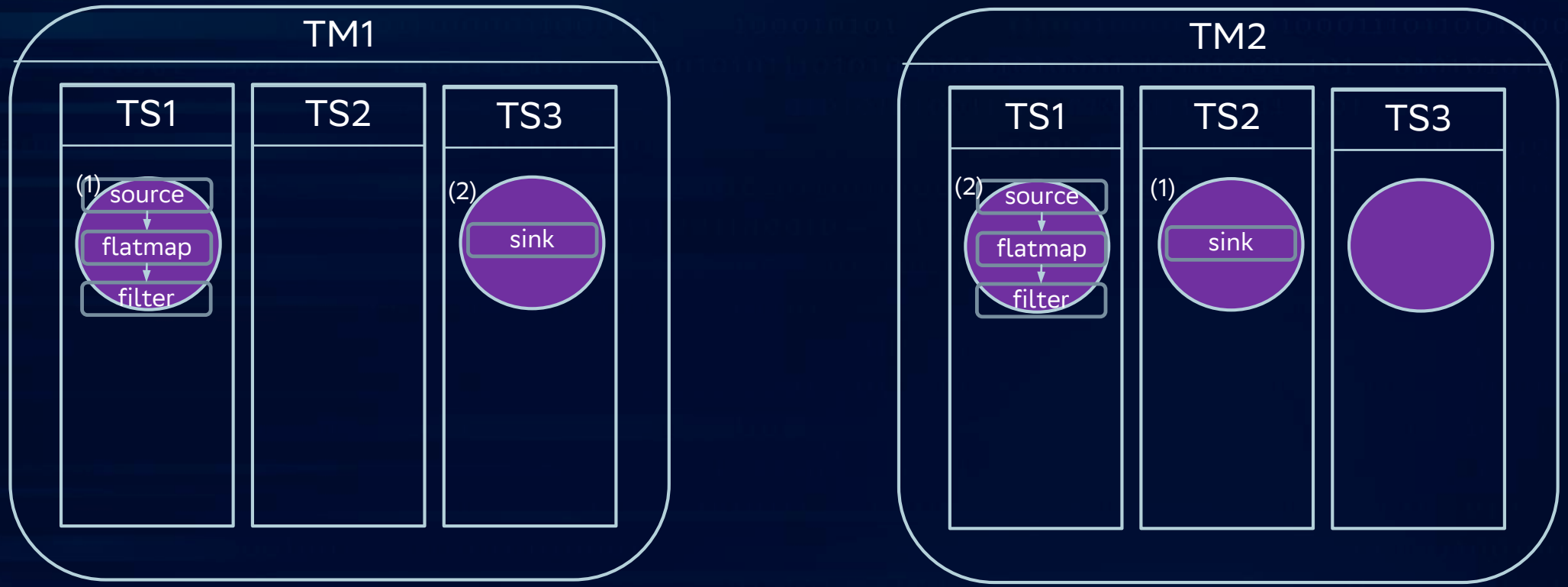
TS = TaskSlot

Distributed execution

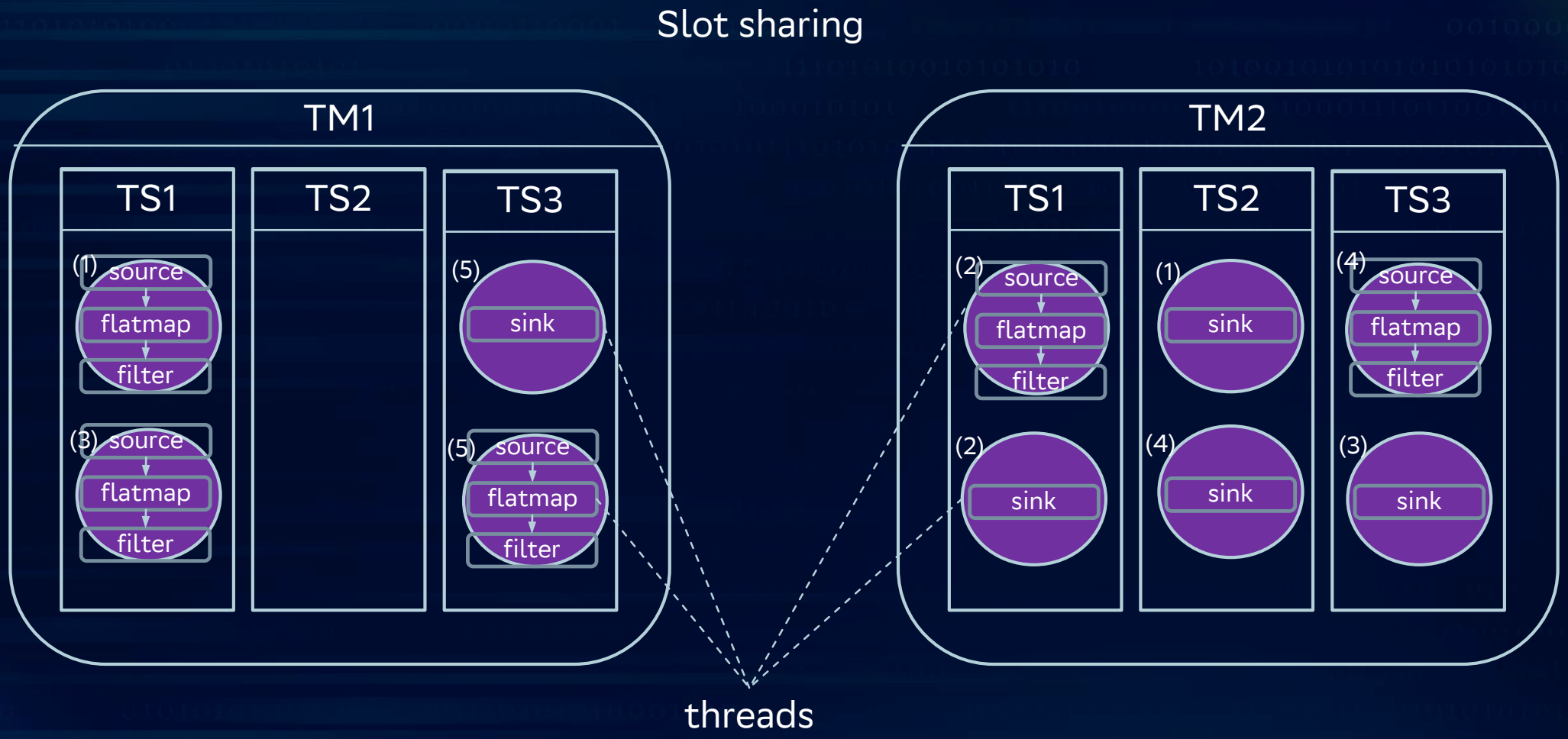
Disable chaining

Distributed execution

Enable chaining

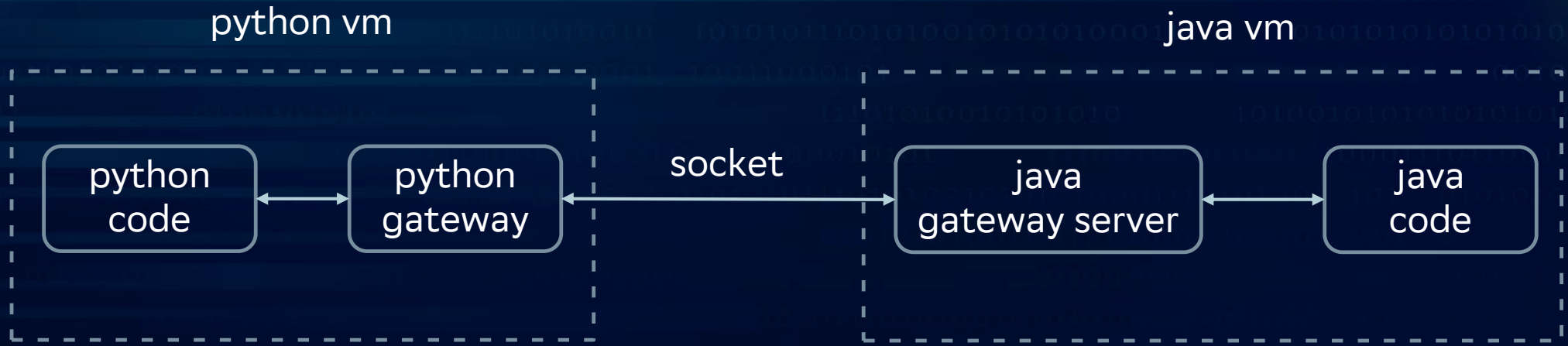


Distributed execution



py4j

spark, flink 1.9-1.14

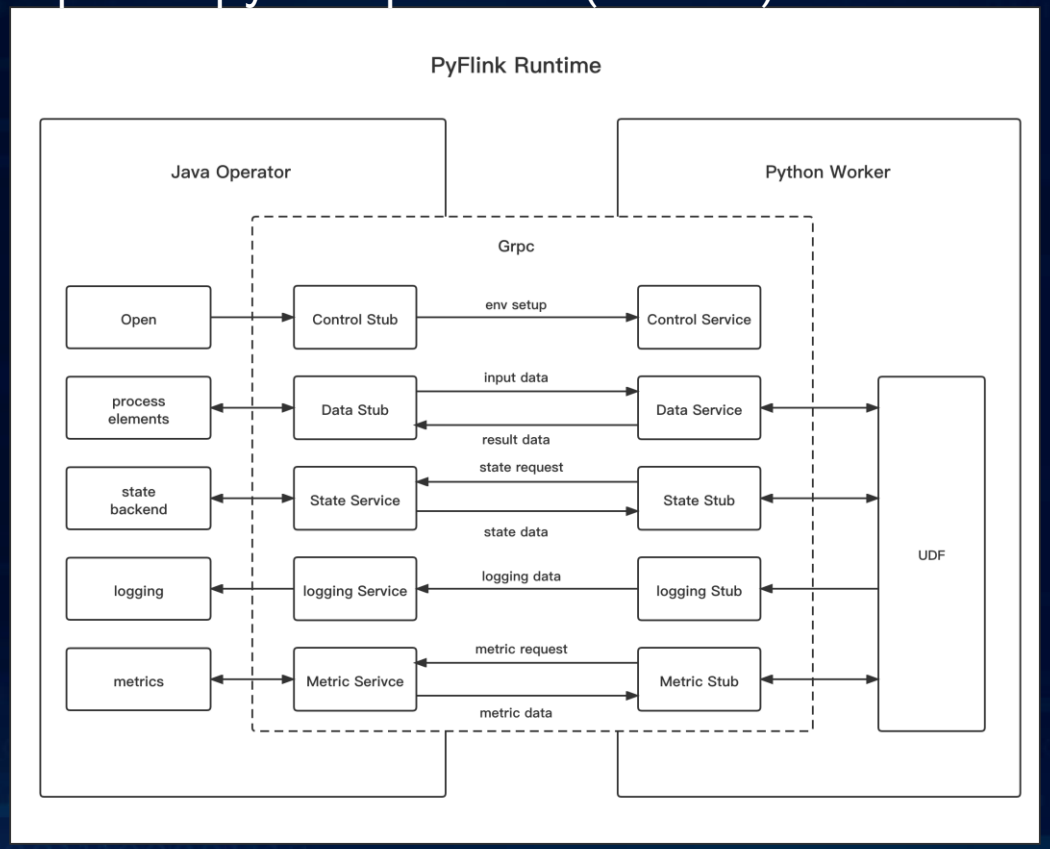


	Py4J	JPype	Jython	PemJa
Связь	Сокеты (tcp)	JNI	Внутри jvm	JNI, shared memory
Процессы	Раздельные процессы	Один процесс	Один процесс	Один процесс
Изоляция	Высокая (крах jvm не убивает python)	Низкая (общая память)	Полная	Низкая (сбой python может привести к падению jvm)
Производительность	Средняя (сокеты)	Высокая	Высокая	Высокая
Ограничения	Блокирующие сокеты	Нет поддержки вызова из jvm	Python2	CPython

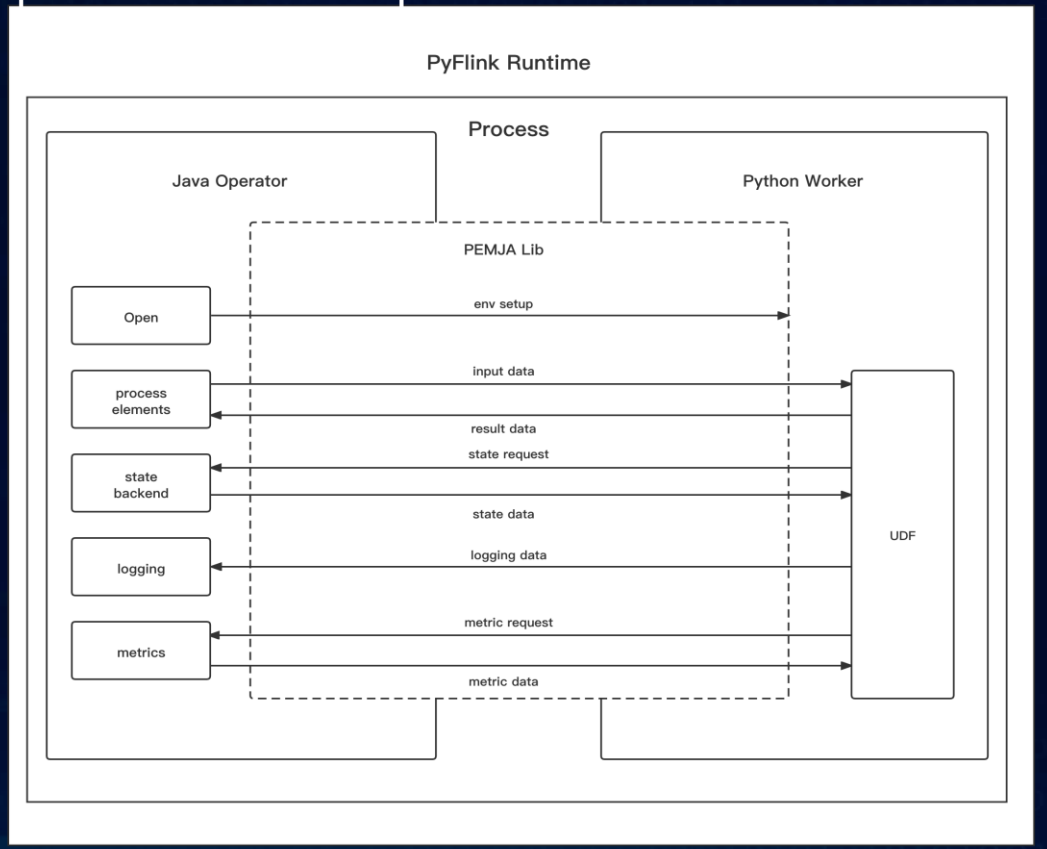
Execution mode

flink 1.15+ pemja (jvm–jni-cpython)

Process execution mode
user-defined functions executed in a separate python process. (default)

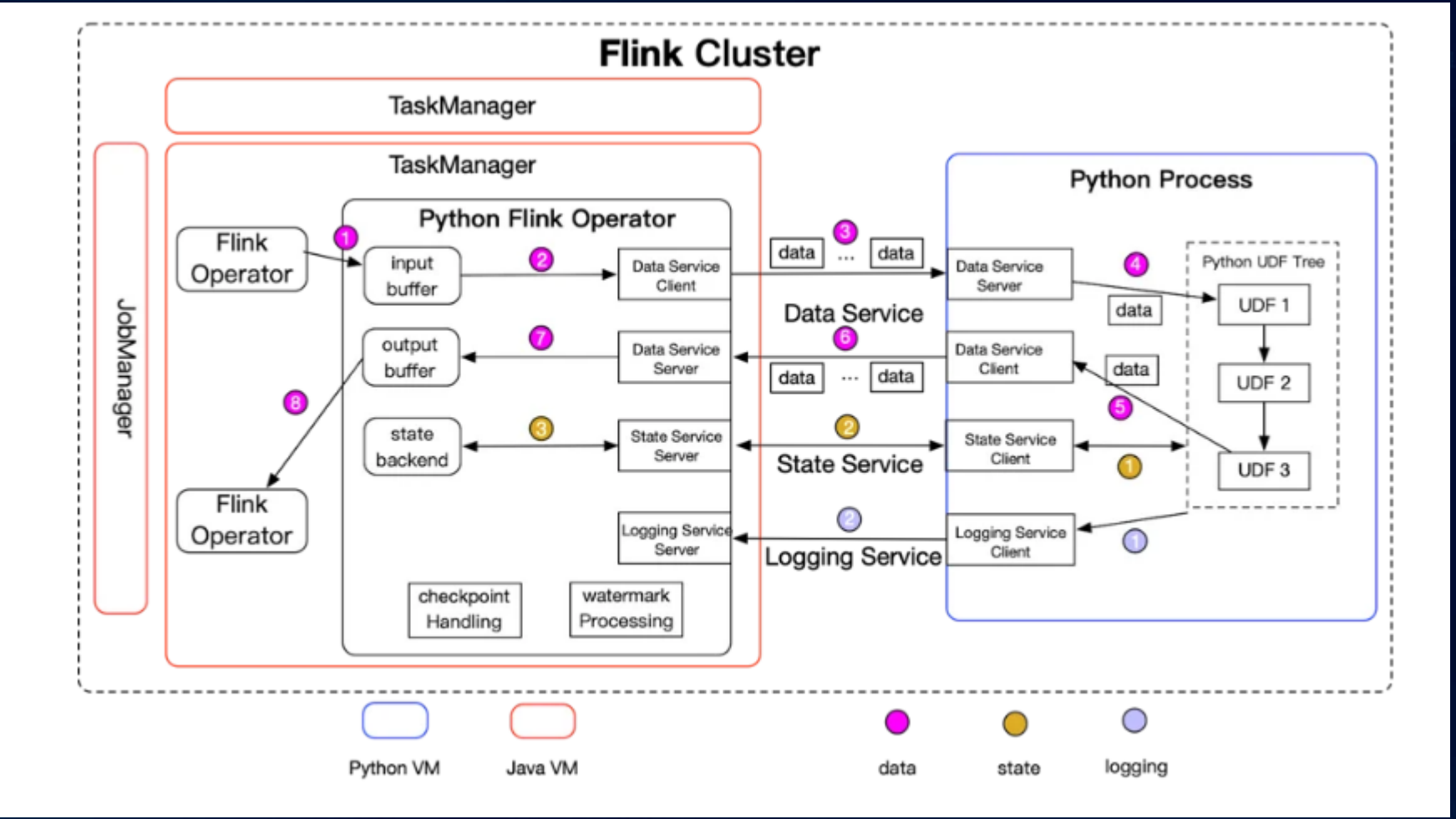


Thread execution mode
user-defined functions executed in the same process as Java operators.



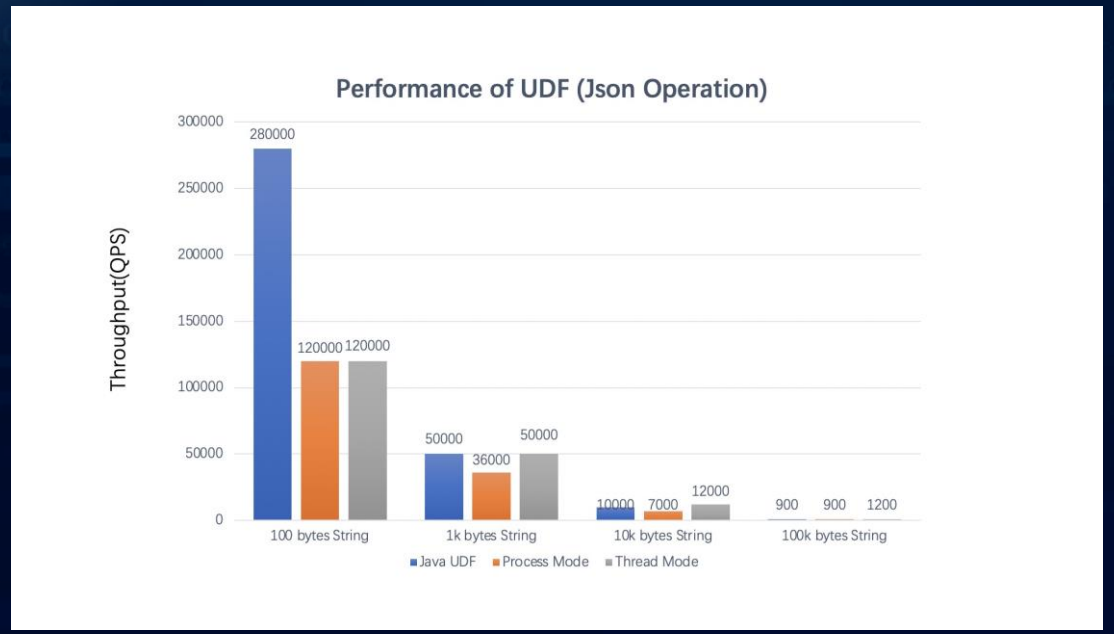
https://nightlies.apache.org/flink/flink-docs-stable/docs/dev/python/python_execution_mode/

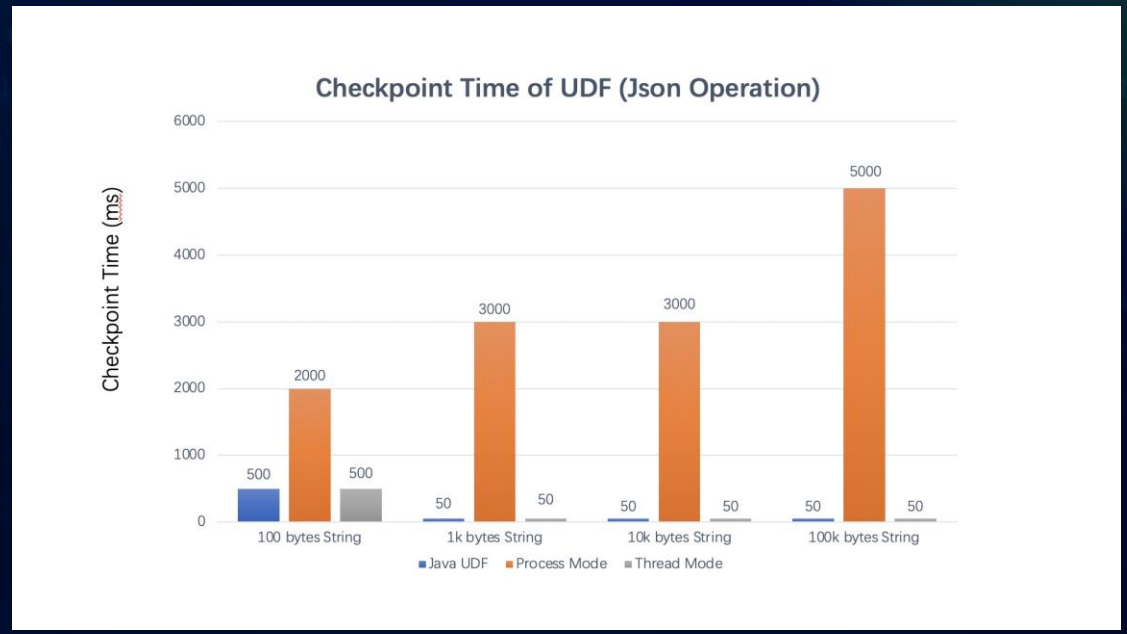
Job execution



https://www.alibabacloud.com/blog/all-you-need-to-know-about-pyflink_600306#:~:text=PyFlink%20reuses%20the%20existing%20job%20compiling%20stack,access%20the%20Java%20objects%20in%20a%20JVM.

Benchmark





OS: Alibaba Cloud Linux (Aliyun Linux) release 2.1903 LTS (Hunting Beagle)
CPU: Intel(R) Xeon(R) Platinum 8269CY CPU @ 2.50GHz
Memory: 16G
CPython: Python 3.7.3
JDK: OpenJDK Runtime Environment (build 1.8.0_292-b10)
PyFlink: 1.15.0

https://flink.apache.org/2022/05/06/exploring-the-thread-mode-in-pyflink/

Рекомендации

Execution mode	Benefits	Limitations
Process mode	• Better resource isolation	• IPC overhead • High implementation complexity
Thread mode	• Higher throughput • Lower latency • Less checkpoint time • Less usage restrictions	• Only support for CPython • Multiple jobs cannot use different Python interpreters in session mode • Performance is affected by the GIL

https://flink.apache.org/2022/05/06/exploring-the-thread-mode-in-pyflink/

pyflink, prepare environment

python -m pip install apache-flink

TableEnvironment

```
env_settings = EnvironmentSettings.in_batch_mode()
t_env = TableEnvironment.create(env_settings)
```

```
#java lib
t_env.get_config().set("pipeline.jars", "file:///jar/path/connector.jar;file:///jar/path/udf.jar")
```

```
#python lib
t_env.add_python_file(file_path)
```

```
#venv
t_env.add_python_archive("venv.zip")
t_env.get_config().set_python_executable("venv.zip/venv/bin/python")
```

```
#python interpreter
t_env.get_config().set_python_executable("/path/to/python")
```

```
t_env.get_config().set("python.execution-mode", "process")
```

```
import logging
logging.warning(table.get_schema())
```

StreamExecutionEnvironment

```
env = StreamExecutionEnvironment.get_execution_environment()
env.set_runtime_mode(RuntimeExecutionMode.BATCH)
```

```
#java lib
env.add_jars("file:///jar/path/connector1.jar", "file:///jar/path/connector2.jar")
```

```
#python lib
env.add_python_file(file_path)
```

```
#venv
env.add_python_archive("venv.zip")
env.set_python_executable("venv.zip/venv/bin/python")
```

```
#python interpreter
env. set_python_executable("/path/to/python")
```

```
conf = Configuration()
conf.set_string("python.execution-mode", “thread")
env = StreamExecutionEnvironment.get_execution_environment(conf)
```

```
import logging
logging.warning(env.get_execution_plan())
```

Типизация

```
ds2 = ds.flat_map(lambda val: [SimpleWord(word, 1) for word in val.split()]
                  , output_type=Types.PICKLED_BYTE_ARRAY()) \
        .key_by(lambda w: w.word, key_type=Types.STRING()) \
        .reduce(lambda w1, w2: SimpleWord(w1.word, w1.cnt + w2.cnt))
```

Output Type

Users could specify the output type information of the transformation explicitly in Python DataStream API. If not specified, the output type will be Types.PICKLED_BYTE_ARRAY by default, and the result data will be serialized using pickle serializer

Data Types

In Apache Flink's Python DataStream API, a data type describes the type of a value in the DataStream ecosystem. It can be used to declare input and output types of operations and informs the system how to serialize elements.

Pickle Serialization

If the type has not been declared, data would be serialized or deserialized using Pickle. For example, the program below specifies no data types.

However, types need to be specified when:

- Passing Python records to Java operations.
- Improve serialization and deserialization performance.

https://nightlies.apache.org/flink/flink-docs-release-2.2/docs/dev/python/datastream/data_types/

https://nightlies.apache.org/flink/flink-docs-release-2.2/docs/dev/python/datastream/operators/overview/

Homework

Настроить окружение (flink, pg, kafka)

Прочитать данные из таблицы-источника

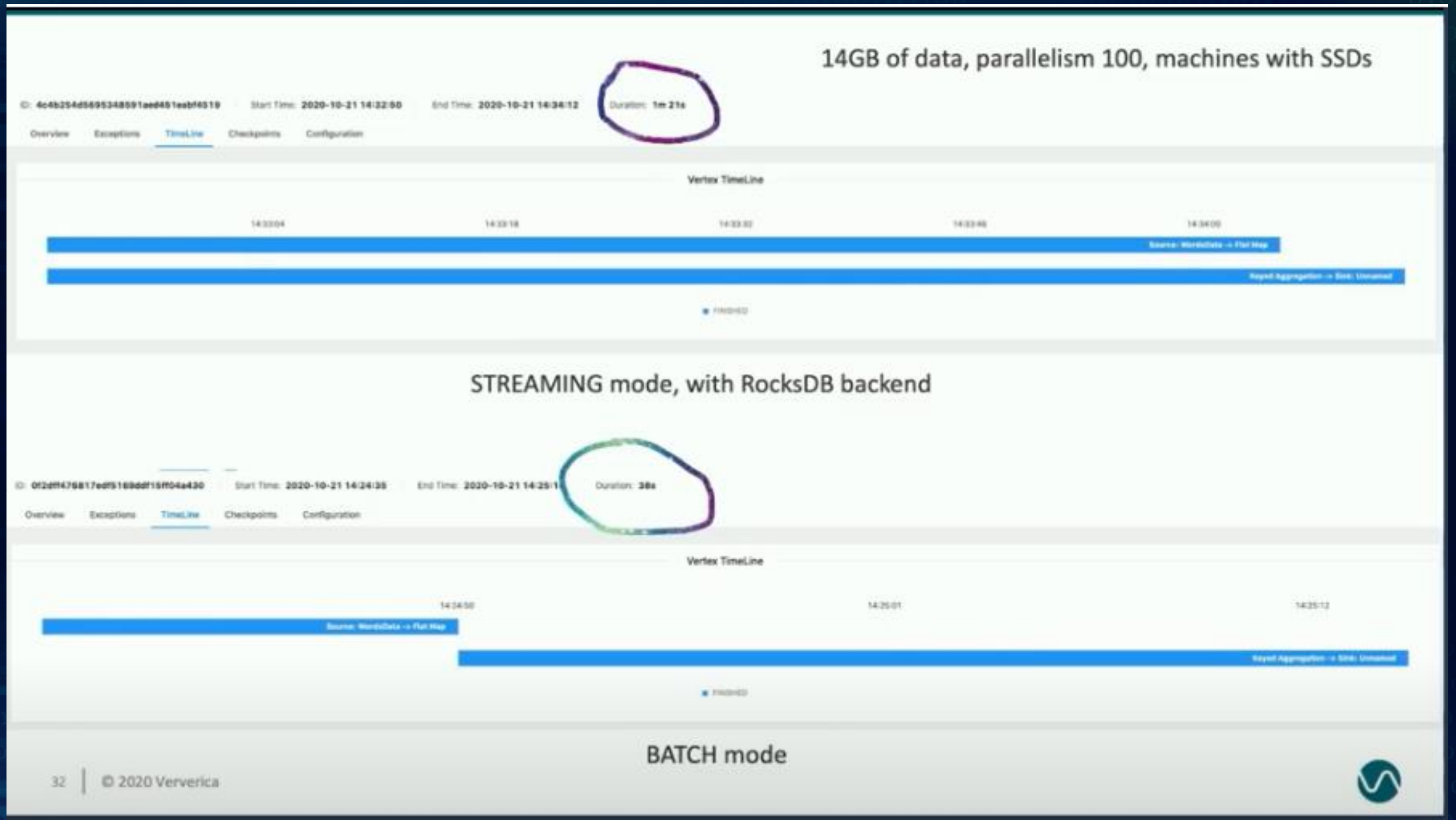
Записать полученные результаты в файл

Questions

?

Спасибо за внимание

Appendix


14GB of data, parallelism 100, machines with SSDs
Overview
Exceptions
TimeLine
Checkpoints
Configuration
Vertex TimeLine
STREAMING mode, with RocksDB backend
Overview
Exceptions
TimeLine
Checkpoints
Configuration
Vertex TimeLine
BATCH mode
32
© 2020 Ververica

SberCode

SBERCODE ACADEMY

Шаг: 1 из 4

Далее

Lesson 3, homework

Prepare postgres

Собираем образ с необходимыми артефактами

```
cd ~ && podman compose build
```

Запускаем контейнер с postgres

```
cd ~ && podman compose up -d postgresql
```

Скрипт создания таблицы customers

Открыть файл ddl.sql

Заходим в контейнер postgressql и создаем таблицу customers

```
cd ~ && podman exec -it postgresql psql -U postgres -d
```

Работа с файлами | Внешние ресурсы

home
ubuntu

Выберите файл в списке слева

Terminal

~$

java example

Batch, wordcount

```
final String filePath = System.getProperty("user.dir") + "\\assets\\data\\words.txt";
final String resPath = System.getProperty("user.dir") + "\\assets\\data\\\\res.txt";
final StreamExecutionEnvironment env = StreamExecutionEnvironment.getExecutionEnvironment();
FileSource<String> src = FileSource.forRecordStreamFormat(
                new TextLineInputFormat(), new Path(filePath))
        .build();
DataStream<String> ds = env.fromSource(
        src,
        WatermarkStrategy.noWatermarks(),
        sourceName: "words");

ds.flatMap(new FlatMapFunction<String, Tuple2<String, Integer>>() {  Serega *
    @Override  Serega
    public void flatMap(String value, Collector<Tuple2<String, Integer>> out) throws Exception {
        for (String word: value.split( regex: " ")) {
            out.collect(new Tuple2<>(word, 1));
        }
}}).keyBy((KeySelector<Tuple2<String, Integer>, String>)  Tuple2<String, Integer> val -> val.f0)
        .sum( positionToSum: 1).print();
```